



Function Approximation

Blink Sakulkueakulsuk

Lecture 1

Agent

Action

Environment

Reward

Next State



Agent

Action

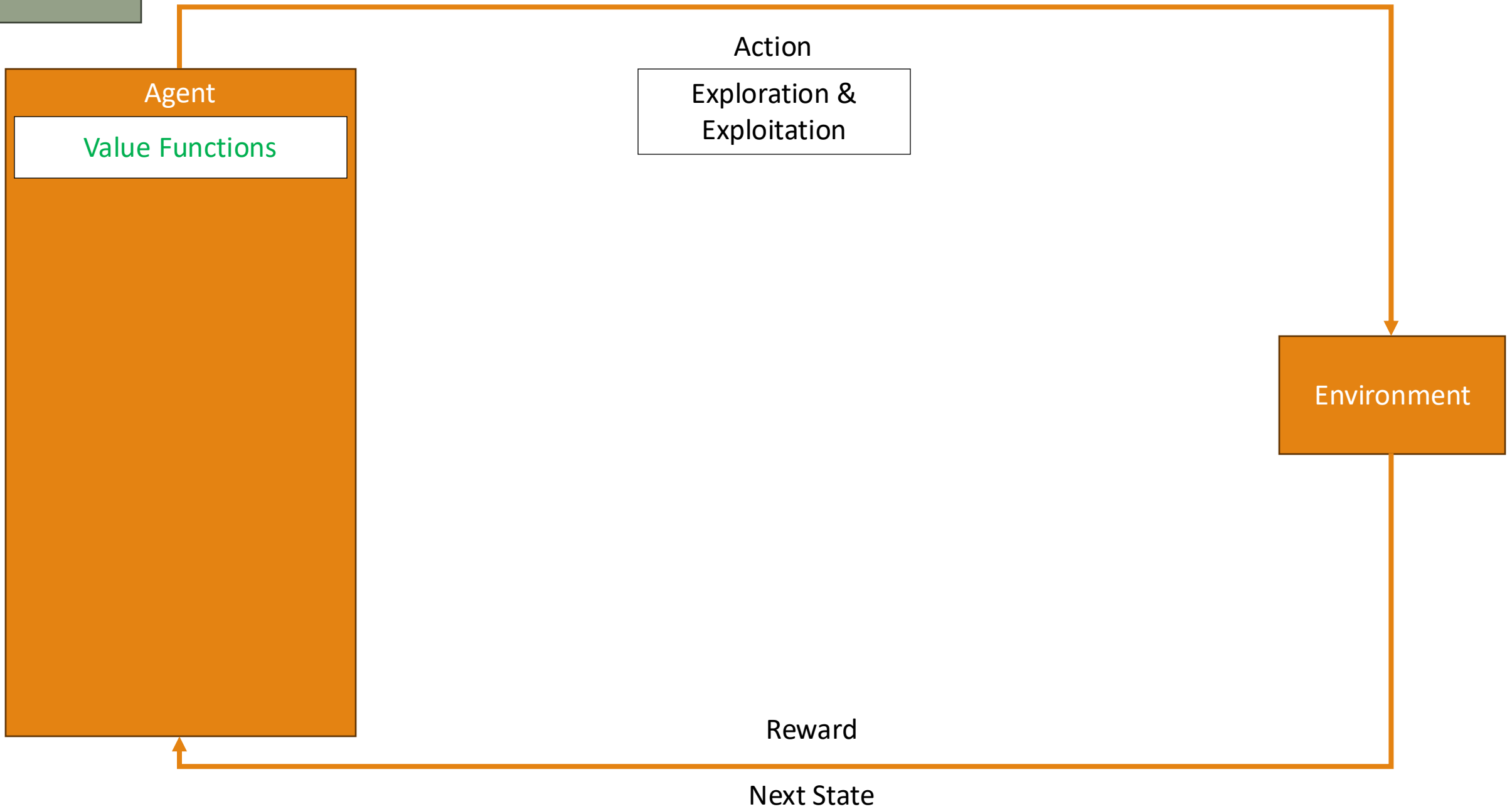
Exploration &
Exploitation

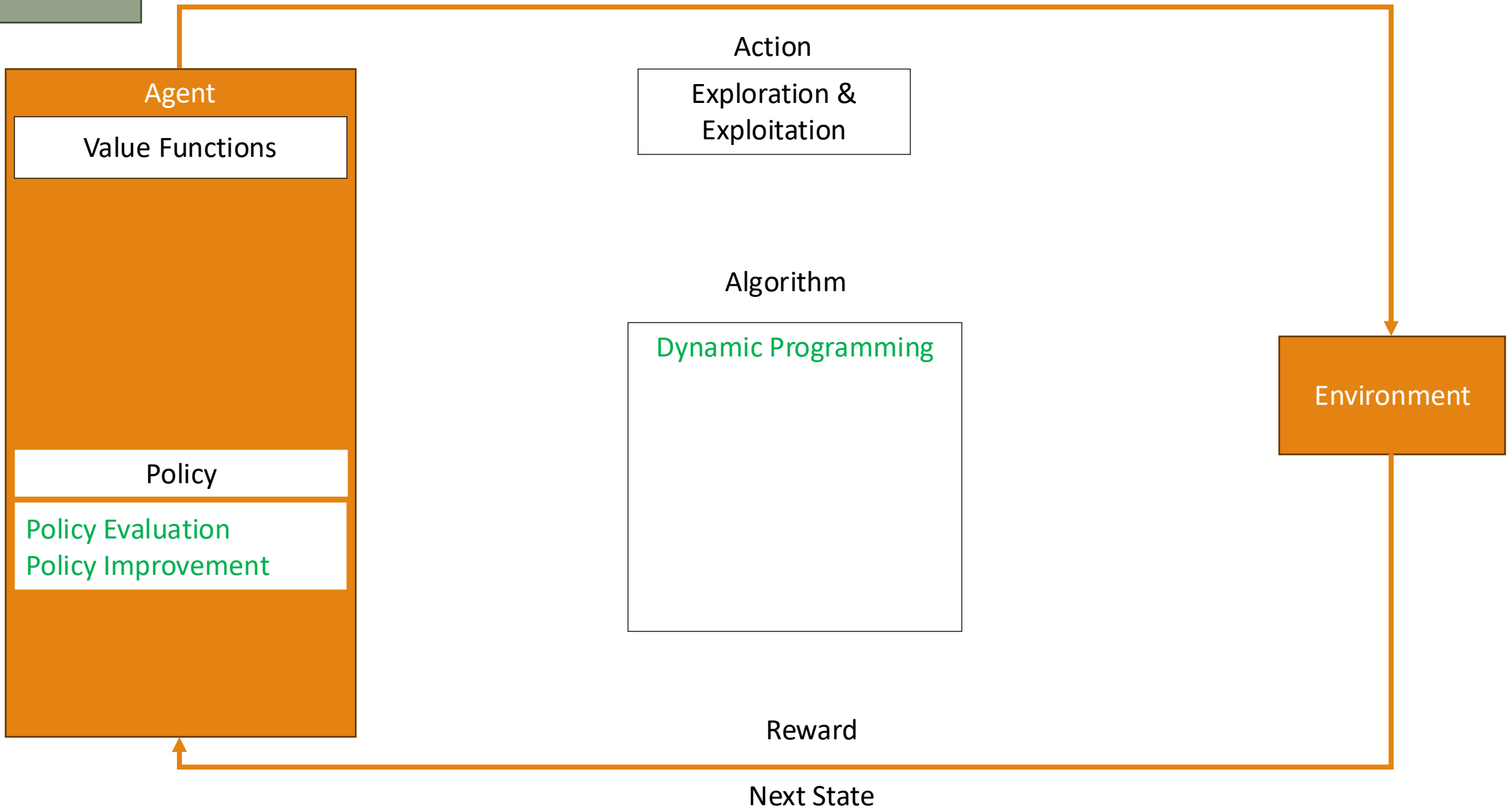
Environment

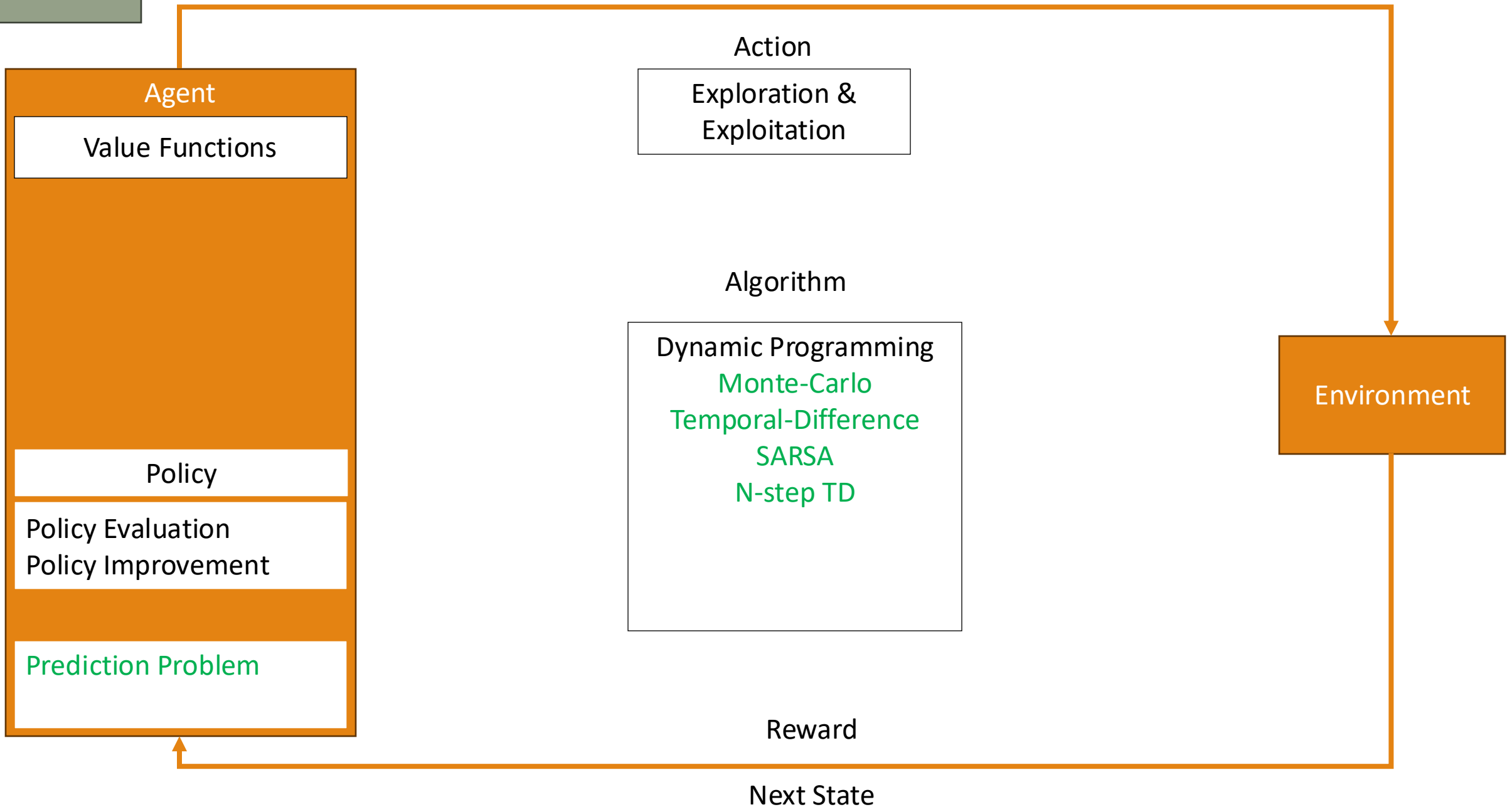
Reward

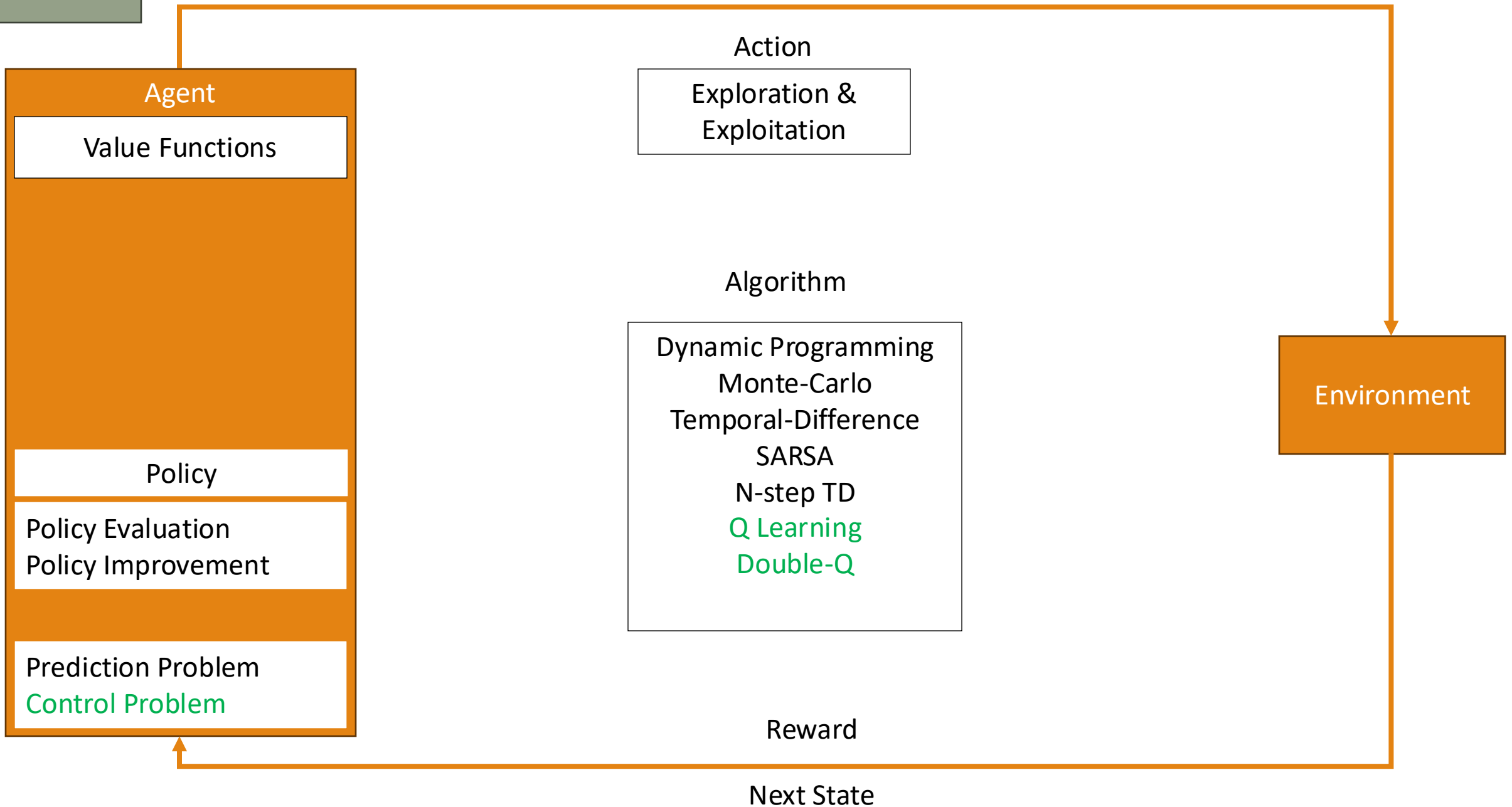
Next State

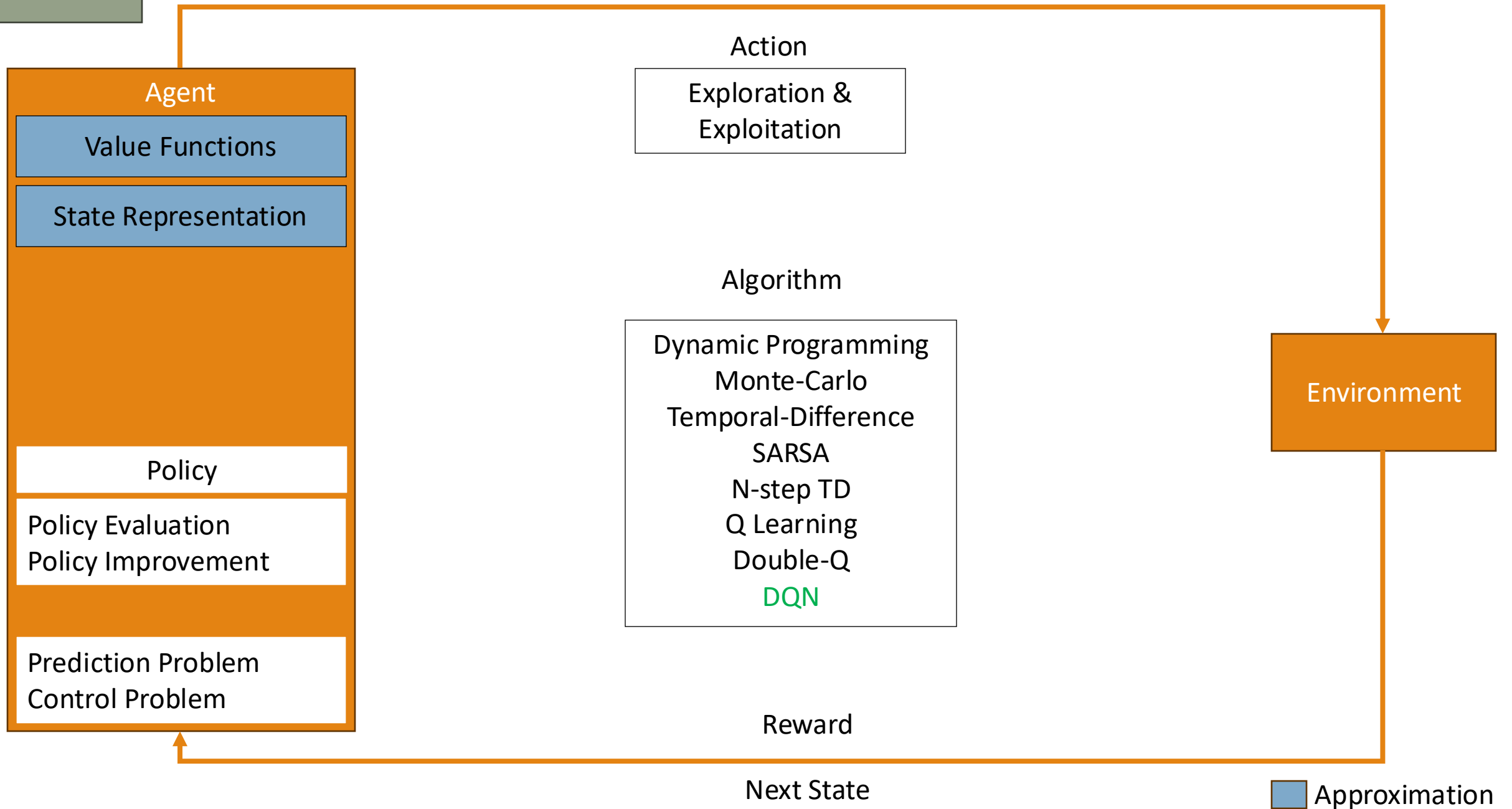












Four main Bellman Equations

Bellman Expectation Equation

$$\begin{aligned}v_{\pi}(s) &= \mathbb{E}[R_{t+1} + \gamma v_{\pi}(S_{t+1}) | S_t = s, A_t \sim \pi(s)] \\q_{\pi}(s, a) &= \mathbb{E}[R_{t+1} + \gamma q_{\pi}(S_{t+1}, A_{t+1}) | S_t = s, A_t = a]\end{aligned}$$

Bellman Optimality Equation

$$\begin{aligned}v^*(s) &= \max_a \mathbb{E}[R_{t+1} + \gamma v^*(S_{t+1}) | S_t = s, A_t = a] \\q^*(s, a) &= \mathbb{E}\left[R_{t+1} + \gamma \max_{a'} q^*(S_{t+1}, a') \mid S_t = s, A_t = a\right]\end{aligned}$$

Prediction and Control

Prediction

- A problem when we perform **policy evaluation**
- Estimating v_π or q_π

Control

- A problem when we perform **policy optimization**
- Estimating v^* or q^*

MC Prediction vs TD Prediction

Prediction problem: learn v_π online from experience under policy π

Monte Carlo:

- Update value $v_n(S_t)$ with respect to sample return G_t

$$v_n(S_t) = v_n(S_t) + \alpha(G_t - v_n(S_t))$$

Temporal-difference learning:

- Update value $v_t(S_t)$ with respect to estimated return $R_{t+1} + \gamma v(S_{t+1})$

$$v_{t+1}(S_t) = v_t(S_t) + \alpha_t(R_{t+1} + \gamma v_t(S_{t+1}) - v_t(S_t))$$

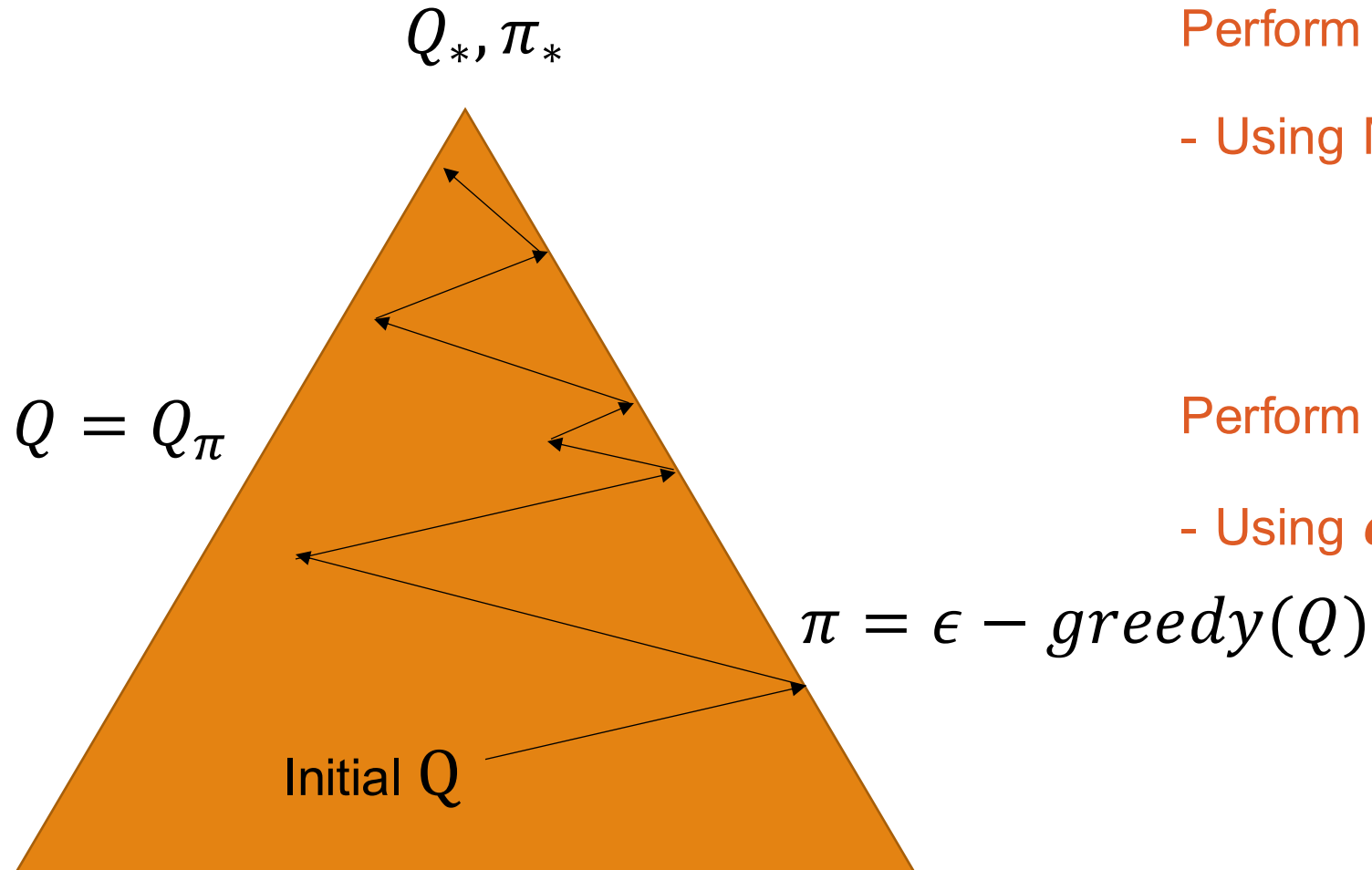
Temporal Difference Learning

We can also learn action values by updating value $q_t(S_t, A_t)$ with respect to estimated return $R_{t+1} + \gamma v(S_{t+1}, A_{t+1})$

$$q_{t+1}(S_t, A_t) = q_t(S_t, A_t) + \alpha_t(R_{t+1} + \gamma q_t(S_{t+1}, A_{t+1}) - q_t(S_t, A_t))$$

This algorithm is called SARSA, because it uses $S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1}$

Monte Carlo Control



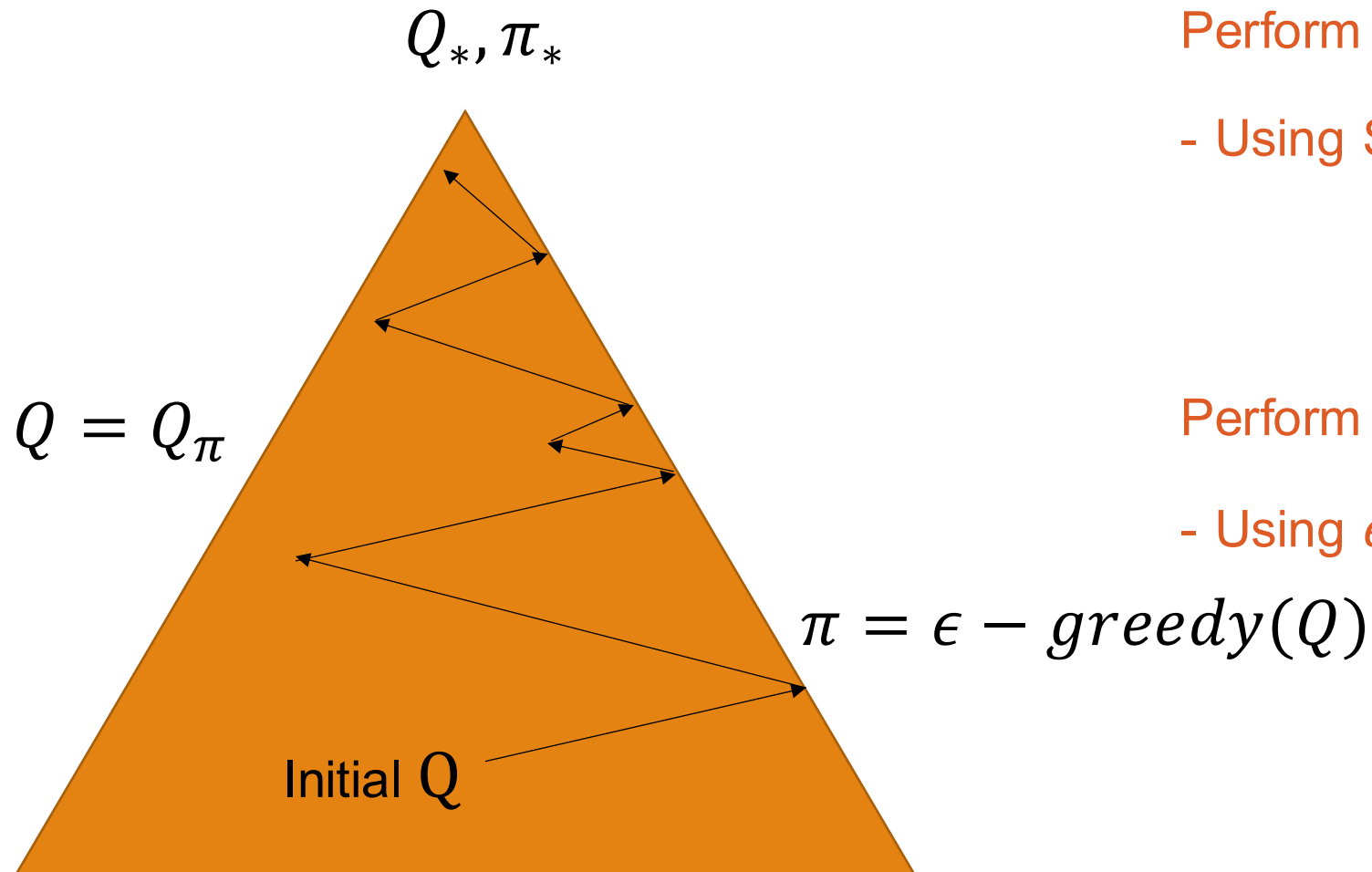
Perform policy evaluation

- Using Monte Carlo policy evaluation
 $q \approx q_\pi$

Perform policy improvement

- Using **ϵ -greedy** policy improvement

SARSA



Perform policy evaluation

- Using SARSA

$$q \approx q_\pi$$

Perform policy improvement

- Using ϵ -greedy policy improvement

On-Policy and Off-policy Learning

On-policy learning

- Learn from actually performing
- Learn a policy π from samples from π

Off-policy learning

- Learn from other sources
- Learn a policy π from samples from b

Q-learning

Q-learning is an off-policy, model-free reinforcement learning

$$q_{t+1}(S_t, A_t) = q_t(S_t, A_t) + \alpha_t(R_{t+1} + \gamma \max_{a'} q_t(S_{t+1}, a') - q_t(S_t, A_t))$$

Q-learning learns from the value of the greedy policy, thus an off-policy learning.

- It converges to the optimal action-value function when we explore all state-action infinitely many times
- No greedy behavior needed

Double Q-learning

Double Q-learning will

- store two q functions: q and q'
- update one q function for each iteration.

$$R_{t+1} + \gamma q'_t(S_{t+1}, \operatorname{argmax}_a q_t(S_{t+1}, a))$$

$$R_{t+1} + \gamma q_t(S_{t+1}, \operatorname{argmax}_a q'_t(S_{t+1}, a))$$

Off-policy Q-Learning Control

To control, we want to optimize both policies.

For the target policy π , we greedify the action on $q(s, a)$

$$\pi(S_{t+1}) = \operatorname{argmax}_{a'} q(S_{t+1}, a')$$

For the behavior policy b , we can use ϵ -greedy on $q(s, a)$

Thus, Q-Learning target is simplified as:

$$R_{t+1} + \gamma \sum_a \pi(a|S_{t+1}) q(S_{t+1}, a) = R_{t+1} + \gamma \max_a q(S_{t+1}, a)$$

Function Approximation

Today

In the past, we deal with discrete domain.

What if we encounter a continuous variable?

Function Approximation

If we look closely, there are a lot of functions in many parts of reinforcement learning.

- policy a function that outputs an action given a state
- value function a function that outputs a value of a state
- agent state update a function that outputs a next state given some inputs
- model a function that outputs dynamic of the environment given some inputs

Function Approximation

All the functions are what we want to learn, but what if the problem space is too large?

- There are too many states, or too many actions

We can approximate these functions.

- This is often called **deep reinforcement learning**, when we use **deep neural network** to represent these functions.

Function Approximation and Large Problem Space

In recent years, deep reinforcement learning is used to solve large problems.

- Go: 10^{170} states
- Drone: continuous state space
- Robots: real world space

Value Function Approximation

So far, we use **lookup tables** to store all values.

- Storing a value $v(S)$ for each state S
- Storing a state-action value $q(S, A)$ for each pair of S, A

In a large problem, this method will run into problems.

- Too many states to store in memory
- Too slow to learn values of states individually
- States may not be fully observable

Value Function Approximation

To solve large MDPs, we can estimate the value function using function approximation instead

$$\begin{aligned}v_{\mathbf{w}}(s) &\approx v_{\pi}(s) \\ q_{\mathbf{w}}(s, a) &\approx q_{\pi}(s, a)\end{aligned}$$

- We can generalize new states (unexplored) from old states (explored)
- We can use MC or TD to learn parameter $\mathbf{w}$.

Agent State Approximation

If we encounter a non-fully observable problem, we can approximate agent state instead.

$$\mathbf{s}_t = u_{\omega}(\mathbf{s}_{t-1}, A_{t-1}, O_t)$$

$\omega \in \mathbb{R}^n$ denotes learnable parameters

$\mathbf{s}_t$ denotes the agent state

- Think of this as a vector of features in a state

Function Approximation Classes

Tabular: A table containing states

State aggregation: Partitioning environment states into discrete states

Linear function approximation

- Fixed agent state update
- Fixed feature map $\mathbf{x} : S \Rightarrow \mathbb{R}^n$
- Linear function as value function approximation $v_{\mathbf{w}}(s) = \mathbf{w}^T \mathbf{x}(s)$

Differentiable function approximation

- $v_{\mathbf{w}}(s)$ is a differentiable function. It can be non-linear.
- We can utilize Deep Learning to learn features, such as CNN.

Function Approximation Classes

Generally, any function approximator works. But reinforcement Learning has some specific properties too keep in mind.

- Experience is not independent and identically distributed (i.i.d.). Current and future actions are correlated.
- Policy affects the observations and rewards
- Learning target can be non-stationary

Policy change, bootstrapping, dynamic of the problem

Function Approximation

How to choose what to use?

Tabular

Good theory, not scalable

Linear function approximation

Good theory, need good feature construction

Non-linear function approximation

Scalable, but not many theories to back up (yet)

Less reliable on picking good features

Gradient Descent Revisited

Given $J(\mathbf{w})$, a differentiable function of parameter vector $\mathbf{w}$

The gradient of $J(\mathbf{w})$ is

$$\nabla_{\mathbf{w}} J(\mathbf{w}) = \begin{pmatrix} \frac{\partial J(\mathbf{w})}{\partial \mathbf{w}_1} \\ \vdots \\ \frac{\partial J(\mathbf{w})}{\partial \mathbf{w}_n} \end{pmatrix}$$

Gradient Descent Revisited

$$\nabla_{\mathbf{w}} J(\mathbf{w}) = \begin{pmatrix} \frac{\partial J(\mathbf{w})}{\partial \mathbf{w}_1} \\ \vdots \\ \frac{\partial J(\mathbf{w})}{\partial \mathbf{w}_n} \end{pmatrix}$$

We want to find the minimum value of $J(\mathbf{w})$. To do that, we can update $\mathbf{w}$ such that it moves in the direction of negative gradient

$$\Delta \mathbf{w} = -\frac{1}{2} \alpha \nabla_{\mathbf{w}} J(\mathbf{w})$$

α is learning rate or step-size parameter.

Approximation using Stochastic Gradient Descent

Given an approximator with parameters $\mathbf{w}$, we want to minimize the value difference between this approximator $v_{\mathbf{w}}(S)$ and true value $v_{\pi}(S)$

$$J(\mathbf{w}) = \mathbb{E}_{S \sim d} [(v_{\pi}(S) - v_{\mathbf{w}}(S))^2]$$

d denotes a distribution of states

So the gradient descent is:

$$\Delta \mathbf{w} = -\frac{1}{2} \alpha \nabla_{\mathbf{w}} J(\mathbf{w}) = \alpha \mathbb{E}_d [(v_{\pi}(S) - v_{\mathbf{w}}(S)) \nabla_{\mathbf{w}} v_{\mathbf{w}}(S)]$$

In sampling algorithm (MC,TD), **Stochastic** gradient descent can be used to sample the gradient.

$$\Delta \mathbf{w} = \alpha (v_{\pi}(S_t) - v_{\mathbf{w}}(S_t)) \nabla_{\mathbf{w}} v_{\mathbf{w}}(S_t)$$

Feature Vectors

If we want to approximate some functions, generally, it relies on states. To make it computable, we represent a state as a **feature vector**

$$\mathbf{x}(s) = \begin{pmatrix} x_1(s) \\ \vdots \\ x_n(s) \end{pmatrix}$$

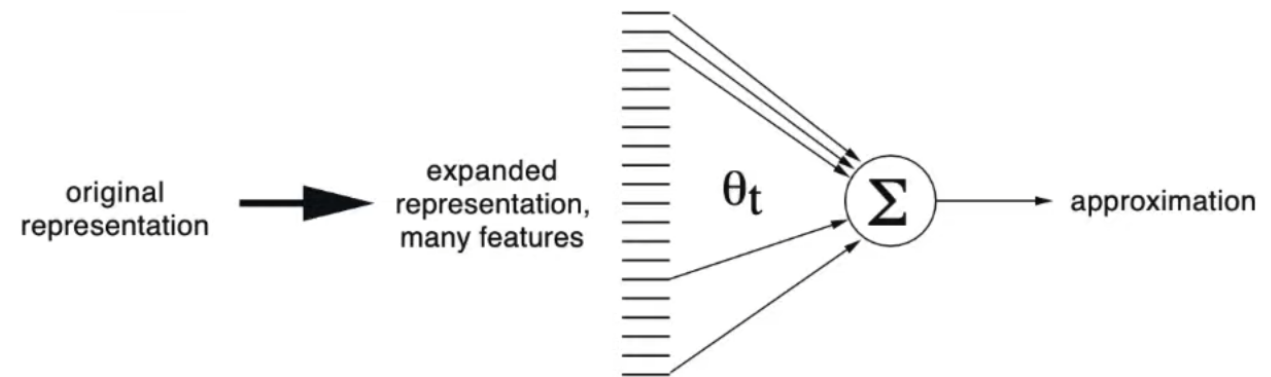
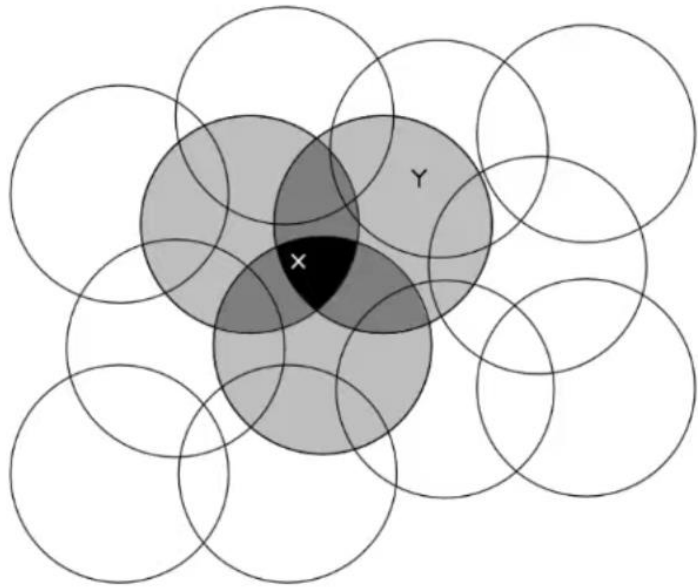
$\mathbf{x}: S \rightarrow \mathbb{R}^n$ is a fixed mapping from state to features

We can represent a feature vector of state S_t as $\mathbf{x}_t = \mathbf{x}(S_t)$

- Distance of a robot from reference points
- Objects in an image

Coarse Coding

Coarse coding is a method to construct a feature vector through state representation.



Linear Value Function Approximation

Let's consider a simple case, a linear function approximator. We can express the value function as

$$v_{\mathbf{w}}(S) = \mathbf{w}^T \mathbf{x}(S) = \sum_{j=1}^n \mathbf{x}_j(S) \mathbf{w}_j$$

The loss of this value function is then defined as

$$J(\mathbf{w}) = \mathbb{E}_{S \sim d} \left[\left(v_{\pi}(S) - \mathbf{w}^T \mathbf{x}(s) \right)^2 \right]$$

Linear Value Function Approximation

$$J(\mathbf{w}) = \mathbb{E}_{S \sim d} \left[\left(v_{\pi}(S) - \mathbf{w}^T \mathbf{x}(S) \right)^2 \right]$$

Then we can use stochastic gradient descent to update the parameter

- Stochastic gradient descent is a gradient descent algorithm that calculates loss of 1 input at a time
- We can update the parameter as

$$\nabla_{\mathbf{w}} v_{\mathbf{w}}(S_t) = \mathbf{x}(S_t) = \mathbf{x}_t \quad \Rightarrow \quad \Delta \mathbf{w} = \alpha \left(v_{\pi}(S_t) - v_{\mathbf{w}}(S_t) \right) \mathbf{x}_t$$

Update = **step-size** **x** **prediction error** **x** **feature vector**

Learning Targets

$$\Delta \mathbf{w}_t = \alpha (v_\pi(S_t) - v_{\mathbf{w}}(S_t)) \nabla_{\mathbf{w}} v_{\mathbf{w}}(S_t)$$

We cannot update $v_{\mathbf{w}}(S_t)$ toward $v_\pi(S_t)$ since we do not know the true value. However, we can change the target according to algorithms we use.

For MC, we use the return G_t

$$\Delta \mathbf{w}_t = \alpha (G_t - v_{\mathbf{w}}(S_t)) \nabla_{\mathbf{w}} v_{\mathbf{w}}(S_t)$$

For TD, we use TD target $R_{t+1} + \gamma v_{\mathbf{w}}(S_{t+1})$

$$\Delta \mathbf{w}_t = \alpha (R_{t+1} + \gamma v_{\mathbf{w}}(S_{t+1}) - v_{\mathbf{w}}(S_t)) \nabla_{\mathbf{w}} v_{\mathbf{w}}(S_t)$$

For TD(λ),

$$\begin{aligned} \Delta \mathbf{w}_t &= \alpha (G_t^\lambda - v_{\mathbf{w}}(S_t)) \nabla_{\mathbf{w}} v_{\mathbf{w}}(S_t) \\ G_t^\lambda &= R_{t+1} + \gamma((1 - \lambda)v_{\mathbf{w}}(S_{t+1}) + \lambda G_{t+1}^\lambda) \end{aligned}$$

Monte-Carlo with Value Function Approximation

The return of MC is an unbiased, noisy sample of $v_{\pi}(s)$

We can use (online) supervised learning to train the model

$$\{(S_0, G_0), \dots, (S_t, G_t)\}$$

So, the linear MC updates is

$$\Delta \mathbf{w} = \alpha(G_t - v_{\mathbf{w}}(S_t)) \nabla_{\mathbf{w}} v_{\mathbf{w}}(S_t) = \alpha(G_t - v_{\mathbf{w}}(S_t)) \mathbf{x}_t$$

MC with linear approximation converges to the global optimum

MC with non-linear value function approximation also converges (to local or global optimum)

TD Learning with Value Function Approximation

The target of TD is a biased sample of $v_\pi(S_t)$

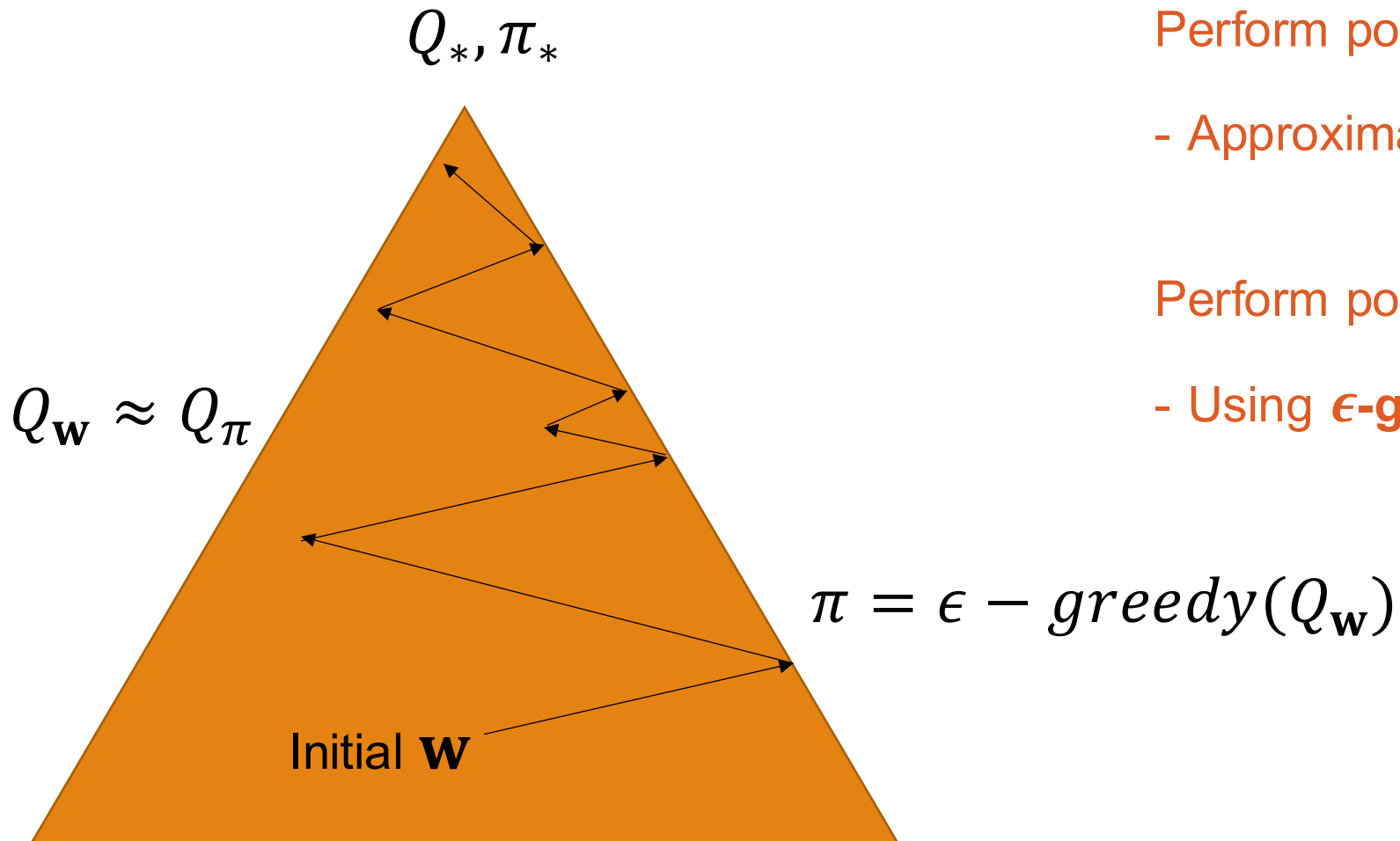
We can still use supervised learning to train the model

$$\{(S_0, R_1 + \gamma v_{\mathbf{w}}(S_1)), \dots, (S_t, R_{t+1} + \gamma v_{\mathbf{w}}(S_{t+1}))\}$$

So, the **linear TD** updates is

$$\Delta \mathbf{w}_t = \alpha (R_{t+1} + \gamma v_{\mathbf{w}}(S_{t+1}) - v_{\mathbf{w}}(S_t)) \nabla_{\mathbf{w}} v_{\mathbf{w}}(S_t) = \alpha \delta_t \mathbf{x}_t$$

Control with Value Function Approximation



Perform policy evaluation

- Approximate policy evaluation

$$q_w \approx q_\pi$$

Perform policy improvement

- Using **ϵ -greedy** policy improvement

Action-Value Function Approximation

We can apply function approximation to action-value function $q_{\mathbf{w}}(s, a) \approx q_{\pi}(s, a)$

For example, the linear action-value function is

$$q_{\mathbf{w}}(s, a) = \mathbf{x}(s, a)^T \mathbf{w}$$

Stochastic gradient descent update is

$$\begin{aligned} \Delta \mathbf{w} &= \alpha (q_{\pi}(s, a) - q_{\mathbf{w}}(s, a)) \nabla_{\mathbf{w}} q_{\mathbf{w}}(s, a) \\ &= \alpha (q_{\pi}(s, a) - q_{\mathbf{w}}(s, a)) \mathbf{x}(s, a) \end{aligned}$$

Deep Reinforcement Learning

We can use deep neural networks as our function approximator

- Apply DNN to approximate value function or action-value function
- Using DNN with MC, TD, Q, or Double-Q

Online Neural Q-Learning

Neural network: $O_t \rightarrow \mathbf{q}_w$

Exploration policy: $\pi_t = \epsilon\text{-greedy}(\mathbf{q}_t)$, then $A_t \sim \pi_t$

Weight update:

$$\Delta \mathbf{w} = k \left(R_{t+1} + \gamma \max_a q_w(S_{t+1}, a) - q_w(S_t, A_t) \right) \nabla_{\mathbf{w}} q_w(S_t, A_t)$$

Optimizer: SGD, Adam,...

Deep Q-Network

Neural network: $O_t \rightarrow \mathbf{q}_w$

Exploration policy: $\pi_t = \epsilon\text{-greedy}(\mathbf{q}_t)$, then $A_t \sim \pi_t$

A replay buffer to store and sample past transitions $(S_i, A_i, R_{i+1}, S_{i+1})$

Target network parameters $\mathbf{w}^-$

Weight update:

$$\Delta \mathbf{w} = \left(R_{t+1} + \gamma \max_a q_{\mathbf{w}^-}(S_{t+1}, a) - q_{\mathbf{w}}(S_t, A_t) \right) \nabla_{\mathbf{w}} q_{\mathbf{w}}(S_t, A_t)$$

Update $\mathbf{w}_t^- \leftarrow \mathbf{w}_t$ periodically (i.e., every 1000 steps)

Optimizer: SGD, Adam,...